DESIGN / MILESTONE

DHT construction over UDP

This milestone implements an always-on manager and an interactive peer. The manager tracks membership; the peers build a directed logical ring and distribute the 14-field storm records into local hash tables.

Submission identity

Group: 70
Contributors: Not yet supplied
Deadline: September 27, 2026, before 11:59 p.m.

Milestone boundary

Implemented manager commands: **register**, **setup-dht**, **dht-complete**. Peer messages: **set-id**, **store**, **stored** and **count-ring**. The last two support reliable completion and reporting. Querying, churn, teardown and graceful deregistration are reserved for the full project.

Process and socket model

The manager accepts one argument: its UDP port. Each peer accepts two: the manager IPv4 address and UDP port. A peer binds its advertised IPv4 address to one m-port for manager exchanges and one p-port for peer exchanges. Commands come from standard input. DHT_GROUP supplies the assigned port range without changing these positional interfaces.

The peer main thread reads commands and waits for manager responses. A separate receiver thread services peer traffic continuously. It never waits for another peer while handling a datagram. The manager processes one datagram at a time, so membership changes are serialized.

Current evidence status

Group 70 is configured. Eight local tests pass, including a four-process UDP run and induced packet loss. The 1950 dataset has also passed a local three-peer run. The two-host demonstration, video upload and actual video timestamps remain pending.

Basis: supplied Socket Project Fall 2026.pdf, sections 1.1, 1.2.1, 2, 3 and 4.1; Canvas assignment "Socket Project: Milestone", assignment 7832035.

Message formats: manager

One UTF-8 JSON object occupies one UDP datagram, with a maximum of 8,192 encoded bytes. JSON escapes strings; no delimiter escaping is required at the application layer. Object key order is not meaningful.

Common envelope

```
v: integer (=1); command: string; request_id: 32 lowercase hex characters
```

Each new request uses a UUID-derived request ID. Retries retain the exact request and ID. Responses copy the ID, use command="response", identify for_command, and contain code="SUCCESS" or "FAILURE". Failures contain reason:string.

register

```
peer_name: string; ip: IPv4 string; m_port: integer; p_port: integer
```

Sent from the peer m-port to the manager port. Names contain 1-15 ASCII letters. All ports are globally distinct and inside the group range. The source endpoint must match ip/m_port. On success, return state="Free". Duplicate names, ports, invalid values or a full registry return FAILURE with no mutation.

setup-dht

```
peer_name: string; n: integer [3,32]; year: integer [1950,2019]
```

Only a registered source can initiate construction. Require no existing DHT and at least n free peers. Select n-1 other peers randomly; mark the requester Leader and the selected peers InDHT. SUCCESS adds peers:[[name,ip,p_port], ...] with leader first, and dht_id:string equal to the setup request ID. The manager enters BUILDING.

dht-complete

```
peer_name: string
```

Sent from the registered leader m-port after all stores and counts are verified. Only the leader of a BUILDING DHT can succeed; SUCCESS adds phase="READY". Other senders or phases fail. New non-completion commands fail while BUILDING. Exact retransmissions are answered from the cache.

Manager state transitions

EMPTY --successful setup-dht--> BUILDING --leader dht-complete--> READY
A second setup fails in BUILDING or READY. Registration normally adds Free peers; while BUILDING, new registrations fail. Malformed envelopes are logged and dropped.

Message formats: peer ring

Every peer request also has the common envelope. dht_id identifies the build session; after initialization, a peer accepts ring messages only from its recorded left neighbor and for the current session.

set-id

```
dht_id: string; id: integer; n: integer; peers: tuple list; s: integer; total: integer; year: integer
```

Leader sends directly to each selected peer before data distribution. The peer validates membership, sets id and right neighbor, and allocates s buckets. SUCCESS/FAILURE uses the common response format. Repeating the same configuration does not reset the table; conflicting configurations fail.

store

```
dht_id: string; seq: integer; pos: integer; owner: integer; record: array of 14 strings
```

The leader sends only to its right neighbor; intermediate peers recompute both hashes and forward to their right neighbor. The owner stores once. seq is the zero-based row index. A store arriving back at ID 0 is an error.

stored

```
dht_id: string; seq: integer; pos: integer; owner: integer
```

Owner copies the store request_id and these routing fields, removes the record, and sends this receipt clockwise. Intermediate peers forward it; the leader matches its pending store. There is no reverse application acknowledgement.

count-ring

```
dht_id: string; counts: array of [id:int, name:string, count:int]
```

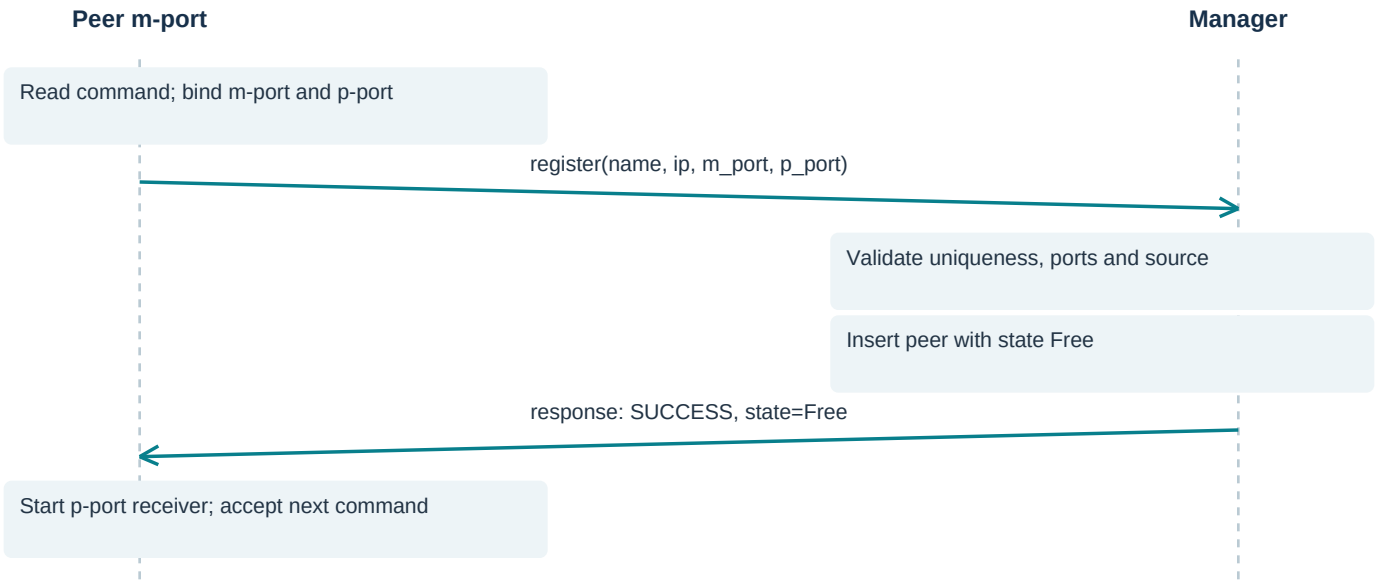
Leader starts counts with its own count. Each peer appends its ID, name and stored count. The token returns clockwise to ID 0 with the same request_id. The leader validates each count against the distribution calculated from the CSV before signaling completion.

record field order

event_id, state, year, month_name, event_type, cz_type, cz_name, injuries_direct, injuries_indirect, deaths_direct, deaths_indirect, damage_property, damage_crops, tor_f_scale. Values are preserved as strings; numeric event IDs are used for hashing. Quoted CSV commas and empty fields are preserved.

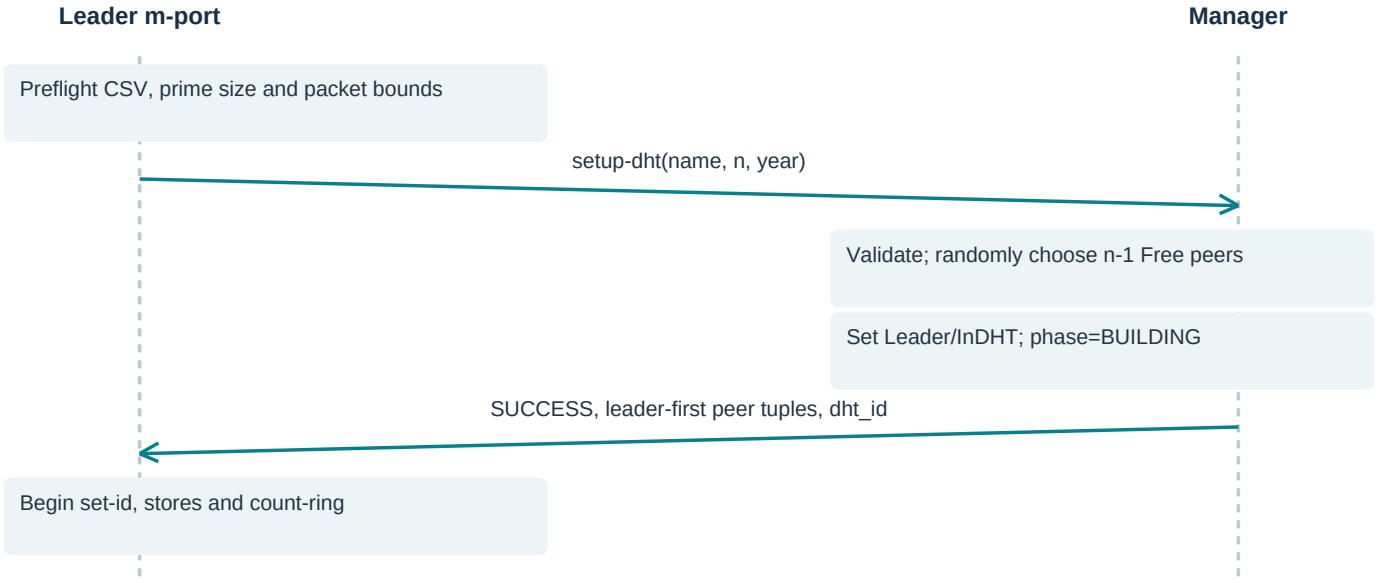
Registration and ring selection

Figure 1. register



Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Figure 2. setup-dht

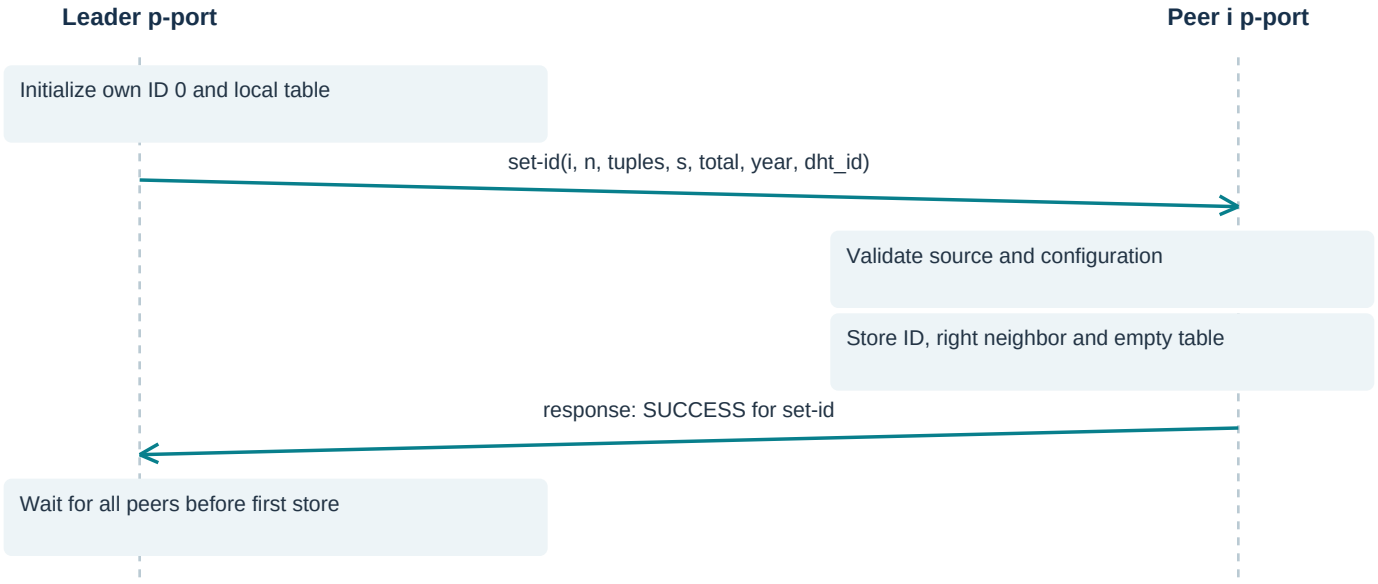


Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Failure paths: the manager returns FAILURE with a reason and does not perform the requested transition. The peer prints COMMAND_FAILED. While BUILDING, only a new valid dht-complete can transition the manager to READY.

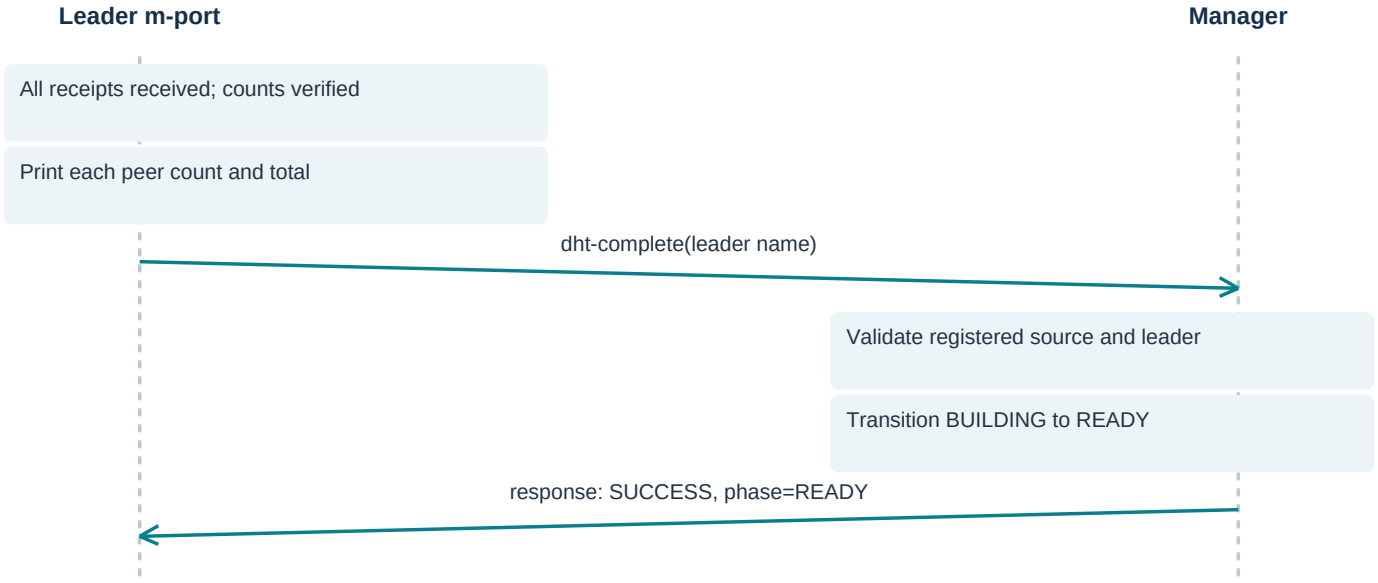
Initialization and completion

Figure 3. set-id (repeat for i = 1 to n-1)



Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Figure 4. dht-complete

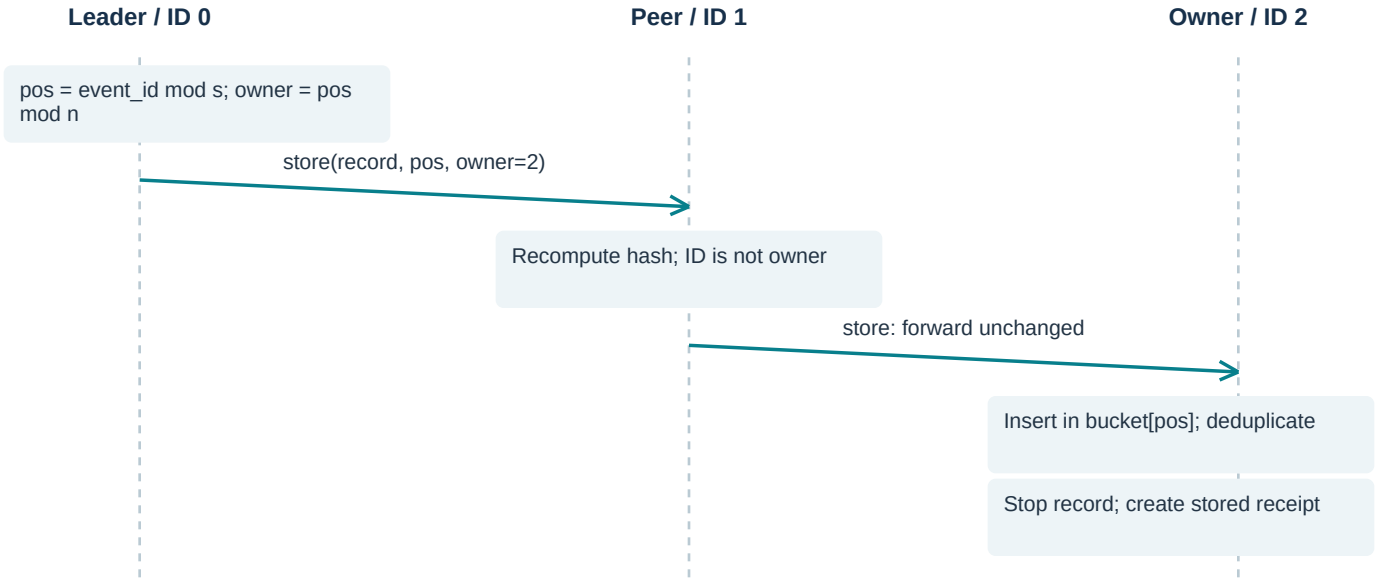


Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Set-id is the initialization exception to clockwise routing. It is completed before any store is issued. The application automatically sends dht-complete after successful verification; stdin cannot force premature completion.

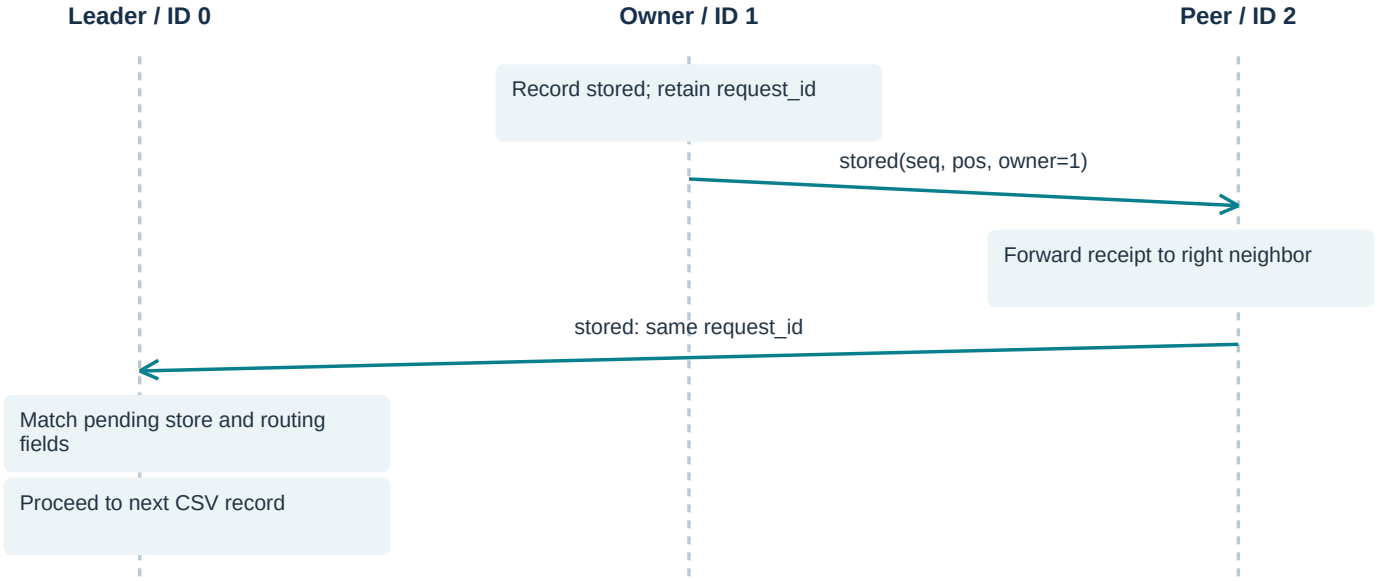
Record placement and receipts

Figure 5. store (example: record belongs to ID 2)



Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Figure 6. stored (example: owner is ID 1)

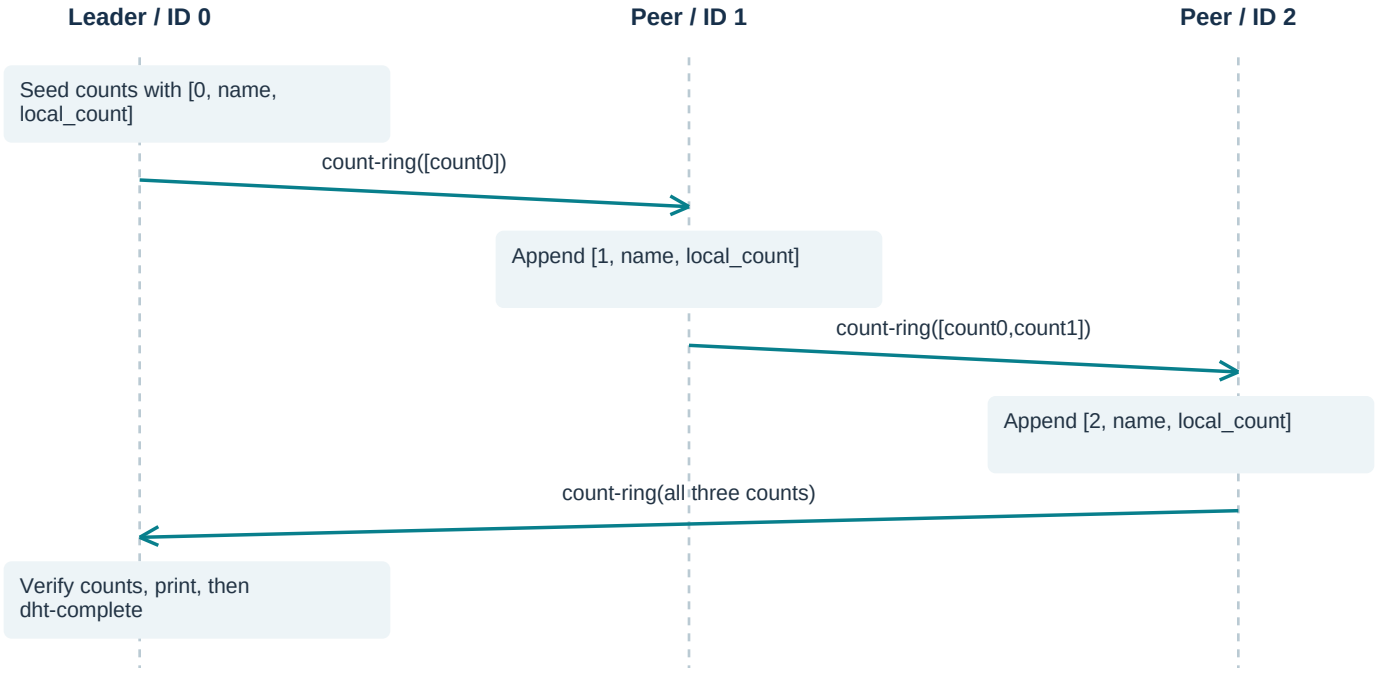


Time flows downward; arrows are UDP messages; shaded boxes are local actions.

For an ID-0-owned record, the leader inserts locally without a store message. For an ID-2-owned record in a three-node ring, its receipt travels directly on the ring edge 2 to 0. The record itself never returns to the leader.

Counting and loss recovery

Figure 7. count-ring



Time flows downward; arrows are UDP messages; shaded boxes are local actions.

UDP retransmission policy

The coordinator waits 0.5 seconds for a matching response or complete ring receipt and retries the exact request up to 10 sends. Manager operations and set-id use direct paired replies. Store and count-ring responses follow the ring, preserving its direction. A lost receipt causes another store traversal, not another stored record.

Manager responses are cached by (source endpoint, request_id), along with the original request bytes. Reusing an ID for a different request fails. The bounded cache retains 4,096 responses. Its guarantee covers retained requests during this run; it is not durable across restarts.

Set-id is idempotent for identical configuration. Local buckets reject a second copy of the same event ID and reject conflicting contents. Count retries start with a fresh leader count and append one entry per peer. Delayed receipts are matched by ID or ignored after the corresponding pending request is removed.

Failure behavior

A persistent timeout leaves the build incomplete, and dht-complete is not sent. Restart the manager and all peers after a partial/uncertain build. There is no rollback or peer-crash recovery in the milestone.

Data structures and validation

Storage and complexity

The manager uses a dictionary keyed by peer name; entries hold IPv4, both ports and Free/Leader/InDHT state. It also holds a ring-name list, leader and phase. Registry lookup is expected O(1); checking port uniqueness and selecting peers costs O(P), where P is registered peers.

Each peer stores an ordered tuple list indexed by ring ID, a configuration/session tuple, and an explicit array of s buckets. The right neighbor is tuples[(id+1) mod n]. Separate chaining preserves distinct events that collide at pos; duplicate checks scan only that bucket.

For l CSV records, s is the first prime strictly greater than 2*l; pos=event_id mod s and owner=pos mod n. Each peer allocates O(s) bucket headers and O(l_i) stored records; the leader additionally holds O(l) input records for preflight and verification. The ring costs O(n) membership space per peer. Local insertion is O(1+b) for bucket size b, worst-case O(l).

One nonlocal record causes exactly n clockwise transmissions in the loss-free case: store messages to its owner and receipts for the remaining path. Thus distribution costs O(l*n) messages, plus initialization and the count pass. Stop-and-wait favors clarity and bounded in-flight state over throughput.

Input and resource decisions

The parser skips one normalized 14-column header, uses csv.reader for quoted fields, and validates year, numeric event IDs and nonnegative injury/death counts. It rejects duplicate event IDs and empty datasets. Bounds are 32 peers, 100,000 records and 8,192 bytes per datagram. Records are serialized in preflight before setup locks the manager.

Both ports bind to the advertised local IPv4 interface. Global port uniqueness follows the assignment conservatively even for peers on different hosts. The group range is [1000+500*G, 1499+500*G]. Peers must be directly reachable; NAT traversal is not included. Session/source checks detect misrouting but do not provide cryptographic authentication.

Observed tests and remaining validation

Eight tests passed: input/schema checks, CSV edge cases, collision preservation, manager rejection/state transitions, retry idempotence and two four-process UDP tests (normal and induced packet loss). The synthetic 40-record fixture yields s=83 and counts [13,14,13]. Actual transmissions were checked against each sender's right neighbor. A separate local run with the 223-record 1950 data reached READY with counts 72, 75 and 76. The two-host demo remains unverified.

Development and demo evidence

Version-control snapshot

The image below is rendered from actual git log output in the local repository; the raw text is included as commit-history.txt. Commits are attributed to Codex (AI assistant), were created during this work and were not backdated. No earlier repository history or private hosted remote is claimed.

```
1ae5026  2026-09-25T11:23:45-07:00
Verify UDP ring under packet loss and document two-host demo procedure
Author: Codex (AI assistant)

cc3dca2  2026-09-25T11:05:55-07:00
Construct peer ring and verify record placement before completion
Author: Codex (AI assistant)

6da57c6  2026-09-25T11:00:42-07:00
Implement UDP message schema and milestone manager state machine
Author: Codex (AI assistant)
```

Video link and actual timestamps

Pending: no video has been recorded or uploaded for this draft.

3a: Compile/check manager and peer programs - not recorded

3b: Run programs on two distinct end-hosts - not recorded

3c: Start manager and register three peer processes - not recorded

3d: Build n=3, YYYY=1950; show counts and dht-complete - not recorded

The planned sequence and narration are in DEMO_GUIDE.md. Planned times are not submitted as actual evidence. The final recording must be at most 7 minutes, have narration, contain no edits/splicing, and be uploaded and accessible to graders before the deadline.

Finalization still required

Add contributors; perform the two-host run; record and upload the continuous demo; enter the real link and timestamps; regenerate/review this PDF; create Socket Project 70.zip.